

Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime

Gianluca Bresolin

gianluca.bresolin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Tommaso Prandin

tommaso.prandin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Abstract

This paper presents the implementation and analysis of a real-time demonstrator originally developed in *Ada* using the *Ravenscar Profile*, re-implemented in *Rust* using the *RTIC* framework. The demonstrator is designed to run on an *STM32F4* microcontroller and uses a *Fixed-Priority Scheduling* policy combined with *Stack Resource Policy (SRP)* for resource sharing. The analysis is performed using the *MAST* tool to verify timing properties and analyze system performance under various workloads. The results demonstrate the feasibility of adopting *Rust* and *RTIC* for real-time applications, while also highlighting differences and considerations compared to the original *Ada* implementation.

CCS Concepts

• **Software and its engineering** → **Real-time systems software**; **Embedded software**; *Software architectures*.

Keywords

Real-Time Kernels, Real-Time Systems, Embedded Software, Software Architectures

ACM Reference Format:

Gianluca Bresolin and Tommaso Prandin. 2025. Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime. In *Proceedings of Real-Time Kernels and Systems Exam (RTKS '25)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Real-time embedded systems demand precise control over timing, concurrency and resource management. Such systems are often used in critical applications where failure to meet timing constraints can lead to severe consequences. Hence, developing a real-time system requires careful consideration of the underlying runtime environment, necessitating a clear understanding of how the system will behave in order to provide sufficient guarantees on meeting timing requirements.

The choice of the runtime environment is a crucial aspect in the design of a real-time system, as it can significantly impact the system's ability to meet its desired behaviour through reliable mechanisms

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RTKS '25, Padova, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

it offers. One of the most popular languages that meets the needs of real-time systems is *Ada*, which has been widely used in the development of safety-critical systems due to its deterministic constructs that allow high analysability of the code. While *Ada* remains a robust choice, recent advances in the *Rust* programming language have introduced new opportunities for real-time systems development.

In this paper, a real-time system originally implemented using the *Ada Ravenscar Profile* is re-implemented in *Rust* using the *Real-Time Intrupt-driven Concurrency (RTIC)* framework, which is responsible for managing the scheduling of tasks through a *Fixed-Priority Scheduling* and the resource sharing in the system under *Stack Resource Policy (SRP)*.

For the analysis of the system, the *MAST* tool is used to verify the timing properties and to analyze the system under different workloads, which is run on a *STM32F4* microcontroller.

The goal of this work is to demonstrate the feasibility of using *Rust* and *RTIC* for real-time systems, while also providing some considerations and comparison with the original *Ada* implementation.

The rest of the paper is organized as follows. Section 2 presents a formal description of the system model. Section 3 introduces the *RTIC* framework and its key concepts. Sections 4 and 5 discuss the measurement of execution times and the detection of deadline misses, respectively. Section 6 provides an overview of the *MAST* tool and its analysis capabilities. In Section 7, we present the *Fixed-Priority Scheduling (FPS)* analysis of the system. Finally, Section 8 concludes the paper reporting the main findings.

2 System Model

The software architecture follows the one proposed in "Guide for the Use of *Ada Ravenscar Profile* in High Integrity Systems" [3]. It is comprised by one periodic producer task that can activate two other sporadic tasks. Additionally, there is a sporadic event server that manages the occurrence of external events. In the implemented application events are generated by a periodic task that emulate user interaction, for repeatability reasons.

Table 1 contains the list of tasks constituting the application, together with their typology, deadline, period (if applicable) and priority. Priorities have been assigned using the *Deadline Monotonic* algorithm, which assigns priorities in decreasing order, starting with jobs of smaller relative deadline. It is an optimal algorithm for constrained deadlines and fixed-priority scheduling [6, p. 119]. The deadline watchdogs have been given the highest priority in order to detect overrun without interference from other tasks. (TODO: is this the proper way?).

The tasks in the system share various resources. Access is regulated by variants of the *Priority Ceiling Protocol* [7] depending on

Task	Task Type	Period (ms)	Deadline (ms)	Priority
Activation Log Reader	Sporadic	3000	1000	3
On Call Producer	Sporadic	5000	800	5
Regular Producer	Periodic	1000	500	7
External Event Server	Sporadic (interrupt)	5000	100	11
Interrupt Generator	Periodic	5000	50	13
ALR Deadline Watchdog	Sporadic	3000	1000	15
OCP Deadline Watchdog	Sporadic	5000	800	15
RP Deadline Watchdog	Periodic	1000	500	15
EES Deadline Watchdog	Sporadic (interrupt)	5000	100	15

Table 1: List of tasks in the system, with their parameters

Resource	Users	Ceiling π_R
Activation Log	Activation Log Reader, External Event Server	11
Event Queue	External Event Server, External Event ISR	11
Request Buffer	Regular Producer, On-Call Producer	7
ALR Deadline Object	Activation Log Reader, ALR Deadline Watchdog	15
OCP Deadline Object	On-Call Producer, OCP Deadline Watchdog	15
RP Deadline Object	Regular Producer, RP Deadline Watchdog	15
EES Deadline Object	External Event Server, EES Deadline Watchdog	15

Table 2: List of shared resources in the system, with their users and priority ceilings

the implementation: the Ada system implements the *Immediate Priority Ceiling Protocol* [4], while the Rust system uses *Stack Resource Policy* [2].

While the implementation is slightly different, both protocols show the same properties and response-time behavior. They require to specify the ceiling value for each shared resource, which is obtained by picking the maximum priority value among all the task using the given resource.

Table 2 contains the list of all the shared resources in the system, together with their users (tasks) and their priority ceiling. (TODO: do we need to add the ISR?)

In order to emulate a realistic workload for task requiring some kind of computation, the small variant of the Whetstone benchmark [5] is used. It includes different kind of computations: integer, floating-point, trigonometry, branches and function calls, with the aim of testing the system under a wide range of loads.

This workload is a parameter that we used in the analysis section to create different workload levels and observe the resulting behavior of the system, first by simulating it using the MAST tool [1], then by running the application in a real setting.

2.1 Tasks Description

Here we provide a brief functional description of each task, in order to give some context to the reader.

2.1.1 Activation Log Reader. The *Activation Log Reader* is a sporadic task which awaits on a signal received from the *Regular Producer*, and then performs its internal operation which consists of an artificial Whetstone workload, followed by a print of the status of the *Activation Log*.

2.1.2 On-Call Producer. The *On-Call Producer* is a sporadic task that waits for a request deposit into the *Request Buffer*, then reads it and runs a Whetstone workload with load equal to the extracted value. The requests are sent by the *Regular Producer*, and they are meant to represent a scenario in which a main task delegates some work to a helper task in the event of an overload.

2.1.3 Regular Producer. The *Regular Producer* is a periodic task that runs a Whetstone workload with a predefined load value, then, based on a predetermined criterion, can release the *Activation Log Reader* and send a work request to the *On-Call Producer*.

2.1.4 External Event Server. The *External Event Server* is a sporadic task that waits for events put onto the *Event Queue* by the interrupt handler bound to the simulated external event. It updates the *Activation Log* counter on each activation.

2.1.5 Deadline Watchdogs. For every task in the system there is a watchdog monitor that checks for deadline overruns. It has the maximum priority to prevent interference (TODO: check). It maintains the number of deadline misses and whenever it detects one it prints a warning message.

Figure 1 shows a high-level view of the system architecture

3 RTIC

4 Execution Times

5 Deadline Miss Detection

In our system, each task is associated with an hard deadline, as described in Table 1. In order to detect a deadline miss, since the *RTIC* framework does not provide any built-in mechanism for this, for each task we implemented a watchdog task responsible for monitoring its deadline.

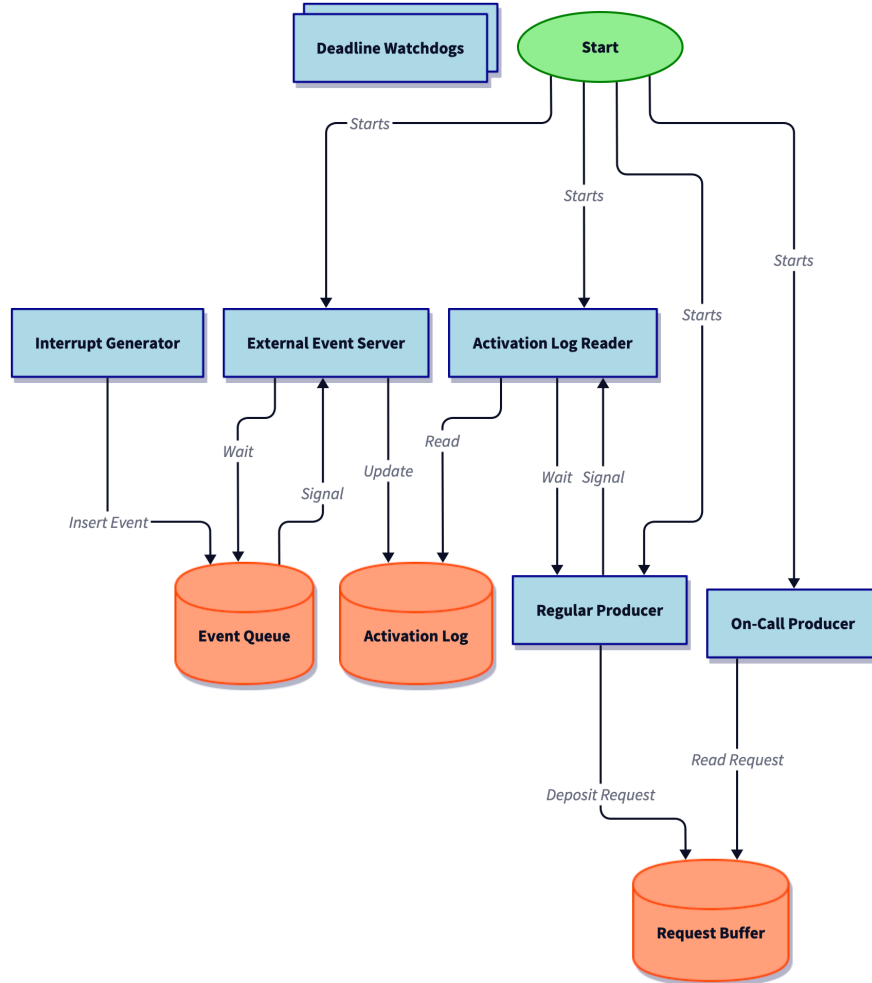


Figure 1: System architecture diagram

To achieve the desired behaviour, i.e., a watchdog declares a deadline miss if and only if the associated task fails to complete its execution before its deadline, we used a shared resource, called `deadline_protected_object`. This resource is a struct containing a boolean flag, which value represents whether the task has completed its execution or not. In *RTIC* framework, each shared resource needs to be acquired before being accessed or before invoking any of its associated methods. Therefore, when a monitored task acquired the lock on the `deadline_protected_object` to invoke `cancel_deadline`, the method responsible for acknowledging the deadline, it can not be preempted by its watchdog task. This prevents the watchdog from preempting the monitored task while it is

updating the flag, avoiding undesired deadline miss detections. Additionally, the `deadline_protected_object` struct contains an activations counter, which allows for finer control over the requests to clear deadline misses, allowing stale requests to be ignored. Since in our system we have periodic and sporadic tasks, we implemented two different types of watchdogs:

- `periodic_deadline_watchdog`: this watchdog has as locals are the period and the next deadline of the monitored task. It delays itself until the next deadline and then, after acquiring the shared `deadline_protected_object`, it invokes on it the method `deadline_miss_detected`. This method checks the flag and, if it is still false, it declares a deadline miss. Then,

the watchdog increments the next deadline by the period of the monitored task and it loops again;

- `sporadic_deadline_watchdog`: this watchdog waits on an activation signal that notices it of a new activation of the monitored task. The watchdog then calculates the next deadline by adding the relative deadline with the activation time of the monitored task, passed as argument inside the precedent signal. The watchdog then delays itself until the obtained absolute deadline and, after acquiring the shared `deadline_protected_object`, it invokes on it the method `deadline_miss_detected`, as happens in the `periodic_deadline_watchdog` case. After that, it loops again, waiting for the next activation signal.

To avoid the first activation of a periodic watchdog from starting before the monitored task can actually execute due to system initialisation, we set its first deadline equal to the system activation instant plus the task's relative deadline. The system activation instant consists in a static `AtomicU32` representing, in ticks, the instant before which all initialization-spawned tasks are delayed: this mechanism provides the system with a well defined "zero time".

Moreover, in order to provide accurate deadline management for sporadic tasks, their activation signals occur inside a critical section. This prevents unwanted preemptions between the task activation and the sending of the signal to the corresponding watchdog, which could otherwise lead to an incorrect calculation of the next deadline.

6 MAST

The response time analysis of the system has been largely performed using the *MAST* tool. *MAST* is a *Modeling and Analysis Suite for Real-Time* applications, developed at the *University of Canababria*. It enables the modelling of real-time systems and provides a set of tools for performing various types of analysis on a given model. In particular, for the purpose of this project, its ability to perform response-time analysis for real-time systems on a single-processor using *Fixed-Priority Scheduling* policy. The tool also supports preemptive tasks and the *Immediate Ceiling Protocol* for managing shared resources.

6.1 MAST Model

To analyze the system using *MAST*, a model of the system must be provided in the form of an XML or text file. In this sub-section, we describe the model components available for representing the system. Further details can be found in the *MAST description*.

6.1.1 Processing_Resource. It represents a resource capable of executing abstract activities. In our case, since we are dealing with a single-processor system, there is only one processing resource. When we define a processing resource, we must provide the range of priorities available, a *System_Timer* and the worst *ISR Switch* time. The latter parameter denotes the context switch time of an interrupt service routine, without taking into account the execution time of the ISR itself.

6.1.2 System_Timer. It represents a timer used by a processing resource. In our system we have to deal with *Ticker*, a system that

has a periodic ticker. The informations required by this type of timer are the period and the worst-case overhead time, i.e. the time taken to process the ticker interrupt, including the scheduling overhead required for scanning and moving the jobs in the ready queue.

6.1.3 Scheduler. It represents the scheduling policy used to manage the amount of CPU processing capacity. As previously mentioned, our system uses a *Fixed-Priority Scheduling* policy. When defining a scheduler, we need to specify the processing resource it is hosted on, the available priority range and the worst context switch time. This last parameter denotes the time taken to set the context switch interrupt as pending and to execute its handler, which is responsible for saving registers of active context and for restoring the ones of the new one.

6.1.4 Shared_Resource. It represents a resource that is shared among tasks. As stated before, the *MAST* model uses the *Immediate Ceiling Protocol*; therefore the priority specified for a shared resource is its ceiling priority, i.e. the highest priority of the tasks that access it.

6.1.5 Scheduling_Server. It represents a schedulable entity in a processing resource. Since we use a *Fixed-Priority Scheduling* policy, the server scheduling parameters are of type *Fixed_Priority_Policy*, which corresponds to regular preemptive fixed-priority scheduling, and require a priority value to be specified.

6.1.6 Operation. It is an abstract representation of a piece of code. There are three types of operations:

- *Simple*: used to model simple operations or methods of protected objects. In the latter case, the operation requires a list of shared resources to lock before executing it and to unlock after its completion. It also requires the specification of best, average and worst-case execution times;
- *Composite*: an operation composed of an ordered sequence of other operations. Its execution times are derived from the execution times of its internal operations;
- *Enclosing*: similar to a composite one but we need to specify the best, average and worst-case execution times as well. This provides a way to express overheads associated with inner operations, such as the cost of acquiring and releasing a shared resource.

6.1.7 Transaction. It represents a modeling unit for interactions of the system. Each transaction is composed of three components:

- *External_Events*: a list of events that models interactions of the system with external devices through interrupts or with hardware timing devices. For each external event it is necessary to specify its type, such as periodic or sporadic, its period or minimum inter-arrival time, its jitter and its phase;
- *Internal_Events*: a list of events that are generated by event handlers. They can be used to express deadlines through *Timing_Requirements*: in our case, we use *Hard_Global_Deadline* timing requirements, which specify an hard deadline that must be met by all instances of the internal event, and a reference to the external event that represents its activation;
- *Event_Handlers*: a list of event handlers that enable to express relationships between events. Each event handler requires an event (or a list of events) as input and produces an event

(or a list of events) as output. Based on the type of the event handler (such as *System_Timed_Activity*), it is also possible to execute an activity operation on a specified activity server.

6.2 MAST Tools

MAST supports different types of analysis tools. In this subsection, we present those used in our system's analysis, while further details can be found in the *MAST description*.

- *Classic RM Analysis*: the classic exact response-time analysis for single-processor fixed-priority systems. Although the name might suggest support only for *Rate Monotonic* policy, it also covers *Deadline Monotonic* as well, which is the policy adopted in our system;
- *Offset-based Approximate with Precedence Relation Analysis*: response-time analysis that improves the holistic analysis by taking into account that tasks of the same transaction are not independent, through the use of offsets. Those precedence relations, together with tasks priorities, provide a tighter estimation of the response times comparing to other tools.

Each analysis tool, as described in the *MAST restrictions*, is subject to certain limitations regarding the expression power of the model, particularly in how transactions can be constructed. The most relevant are:

- *Restricted_Multipath_Transactions*: it imposes the absence of branch elements in side transactions;
- *Referenced_Events_Are_External_Only*: it requires only external events as referencable events, therefore only external events can be used as activation events for deadlines;
- *Linear_Transactions_Only*: it allows just one external event per transaction.

7 FPS Analysis

In this section we present our *Fixed-Priority Scheduling (FPS)* analysis of the system. The analysis was performed by modeling the system through a *MAST* model, which was then analyzed using the *MAST* tool to verify the timing properties and to analyze the system under different workloads.

The priority assignment was done according to the *Deadline Monotonic* policy, which assigns higher priorities to tasks with shorter deadlines. This choice comes from the fact that tasks in the system do not have implicit deadlines and, as theory proves, *Deadline Monotonic* performs better than *Rate Monotonic* in such cases, as it can provide feasible schedules where *Rate Monotonic* fails.

The evaluation of the system under different utilization levels was facilitated by the use of the *Small Whetstone* algorithm to simulate computational workloads. Thanks to its configurable load and deterministic behaviour without involving the use of shared resources, it provides a straightforward way to test the system under different utilizations by simply scaling the WCET in the different *small_whetstone* operations that are present in the *MAST* model, without the need for new execution-time measurements.

7.1 Holistic Analysis

We first conducted an holistic analysis, considering all the tasks as independent. As a consequence, this analysis is pessimistic, since it

does not take into account the relationships between tasks. System feasibility is therefore evaluated based on the worst-case scenario at the critical instant, i.e., when all tasks are released simultaneously (not a realistic and possible case). The analysis was performed using the *Classic RM Analysis* tool in *MAST*, which implements the classic exact response-time analysis for single-processor systems under *Fixed-Priority Scheduling* (although it's called *Rate Monotonic*, it bases *Deadline Monotonic* analysis).

The first results of the analysis were made considering the system under the loads provided in the original *Guided Extended Example*, i.e., with the following Whetstone workloads (accompanied by their corresponding WCETs expressed in seconds).

Task	Workload	Rust WCET	Ada WCET
Regular_Producer	756	0.023639364	0.00554170
On_Call_Producer	278	0.008692782	0.00203782
Activation_Log_Reader	139	0.004346391	0.00101891

Table 3: Whetstone Operations

The obtained results are reported in Table [4] and Table [8]. Times are expressed in seconds and we omitted the watchdogs' transactions (which were instead included in the Rust model) since they are not relevant for identifying increasing workload factors. Regarding the jitter times, all transactions suffer jitter due to the system ticker interrupt and the context switch overhead. Moreover, additional sources of jitter and blocking times for each transaction are the following:

- *rp_transaction*: suffers additional jitter due to the possible interference of the *ees_transaction*. Its blocking time is instead due to the possible access to shared resources from the lower-priority *ocp_transaction* and *alr_transaction* with ceiling priority equal to the priority of *rp_transaction*;
- *ocp_transaction*: suffers additional jitter due to interference by the higher-priority *rp_transaction*. Its blocking time is caused by the *alr_transaction*;
- *alr_transaction*: suffers additional jitter due to interference by both the higher-priority *rp_transaction* and *ocp_transaction*. It does not suffer blocking, since it represents the lowest-priority task in the system;
- *ees_transaction*: it suffers no additional jitter than the aforementioned overheads. Its blocking time is caused by the *alr_transaction* which can possibly access the shared *Activation_Log* resource with ceiling priority equal to the priority of *ees_transaction*.

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.024728	2006.3%	0.001066	3.030E-06
ocp_transaction	0.033451	8791.0%	0.024741	2.730E-06
alr_transaction	0.037822	21927.3%	0.033455	0.000
ees_transaction	0.001043	>=100000.0%	2.955E-05	2.730E-06
System Utilization	2.79%			
System Slack	1994.1%			

Table 4: Holistic Analysis Results Rust

Based on these results, since the smallest slack available was 2006.3% for the `rp_transaction`, we increased the workloads with a factor of 20 to reduce it significantly. New results are reported in Table [5].

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.474326	5.08%	0.001516	3.030E-06
<code>ocp_transaction</code>	0.648377	87.11%	0.474643	2.730E-06
<code>alr_transaction</code>	0.735413	303.91%	0.787757	0.000
<code>ees_transaction</code>	0.001043	>=100000.0%	0.001030	2.730E-06
System Utilization	53.76%			
System Slack	5.75%			

Table 5: Holistic Analysis Results Rust x20

We can notice how the blocking times have not changed, since protected operations remain the same. While we have achieved a small slack for the `rp_transaction` (5.08%), the other two transactions that involve Whetstone computations still have large available slacks. Based on these results, especially on the smallest slack of 87.11%, we decided to increase the `On_Call_Producer` and `Activation_Log_Reader` workloads by a factor 1.8, while keeping the `Regular_Producer` workload fixed. The updated results are reported in Table [6].

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.474326	2.34%	0.001516	3.030E-06
<code>ocp_transaction</code>	0.787600	3.91%	0.473643	2.730E-06
<code>alr_transaction</code>	0.944248	35.55%	0.786757	0.000
<code>ees_transaction</code>	0.001043	91855.5%	0.001030	2.730E-06
System Utilization	58.86%			
System Slack	1.58%			

Table 6: Holistic Analysis Results Rust x1.8

Finally, based on the last useful available slack of 35.55% for the `alr_transaction`, we increased its workload by a factor of 1.35. The updated results are reported in Table [7].

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.474326	0.00%	0.001516	3.030E-06
<code>ocp_transaction</code>	0.787600	0.00%	0.474643	2.730E-06
<code>alr_transaction</code>	0.999068	0.39%	0.787812	0.000
<code>ees_transaction</code>	0.001043	6911.7%	0.001030	2.730E-06
System Utilization	60.68%			
System Slack	0.39%			

Table 7: Holistic Analysis Results Rust x1.35

Even though the `ees_transaction` has still a large available slack, since it does not perform any Whetstone computation, it is not possible to increase the utilization of the system any further. Besides that, even an eventual increase of the execution time of `ees_transaction` would not affect significantly the utilization of

the system. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 60.68%.

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.006624	8808.2%	0.001045	1.361E-06
<code>ocp_transaction</code>	0.007697	38006.3%	0.005638	1.267E-06
<code>alr_transaction</code>	0.008752	91394.9%	0.007706	0.000
<code>ees_transaction</code>	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	1.03%			
System Slack	7728.7%			

Table 8: Holistic Analysis Results Ada

Based on these results, since the smallest available slack was 8808.2% for the `rp_transaction`, we increased the workloads with a factor of 90 to reduce it significantly. The updated results are reported in Table [9].

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.490670	1.56%	0.002963	1.361E-06
<code>ocp_transaction</code>	0.669740	72.27%	0.490390	1.267E-06
<code>alr_transaction</code>	0.759792	265.23%	0.670102	0.000
<code>ees_transaction</code>	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	55.74%			
System Slack	1.98%			

Table 9: Holistic Analysis Results Ada x88

Again, we can notice how the blocking times have not changed, since protected operations remain the same. While we have achieved a small slack for the `rp_transaction` (1.56%), the other two transactions that involve Whetstone computations still have large available slacks. Based on these results, especially on the smallest slack of 72.27%, we decided to increase the `On_Call_Producer` and `Activation_Log_Reader` workloads by a factor of 1.7, while keeping the `Regular_Producer` workload fixed. The updated results are reported in Table [10].

Transaction	R_{max}	Slack	Jitter	MBT
<code>rp_transaction</code>	0.490670	0.78%	0.002963	1.361E-06
<code>ocp_transaction</code>	0.795769	1.17%	0.490889	0.000
<code>alr_transaction</code>	0.948836	32.42%	0.796380	1.361E-06
<code>ees_transaction</code>	1.882E-05	56921.9%	1.141E-05	1.267E-06
System Utilization	60.35%			
System Slack	0.78%			

Table 10: Holistic Analysis Results Ada x1.7

Finally, based on the last useful slack available of 32.34% for the `alr_transaction`, we increased its workload by a factor of 1.3. The updated results are reported in Table [11].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.490670	0.78%	0.002963	1.361E-06
ocp_transaction	0.795769	1.17%	0.490889	0.000
alr_transaction	0.994747	1.95%	0.796563	1.361E-06
ees_transaction	1.882E-05	56921.9%	1.141E-05	1.267E-06
System Utilization	61.87%			
System Slack	0.78%			

Table 11: Holistic Analysis Results Ada x1.3

Again, the considerations made before upon the `ees_transaction` apply here. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 61.87%.

8 Conclusions

9 Citations and Bibliographies

References

- [1] [n. d.]. MAST: A Comprehensive Tool Suite for Real-Time Systems Analysis and Optimization – ACM SIGBED.
- [2] T. P. Baker. 1991. Stack-Based Scheduling of Realtime Processes. *Real-Time Systems* 3, 1 (March 1991), 67–99. doi:10.1007/BF00365393
- [3] Alan Burns, Brian Dobbing, and Tullio Vardanega. 2004. Guide for the Use of the Ada Ravenscar Profile in High Integrity Systems. *Ada Lett.* XXIV, 2 (June 2004), 1–74. doi:10.1145/997119.997120
- [4] Albert M. K. Cheng and James Ras. 2007. The Implementation of the Priority Ceiling Protocol in Ada-2005. *Ada Lett.* XXVII, 1 (April 2007), 24–39. doi:10.1145/1274610.1274611
- [5] H. J. Curnow and B. A. Wichmann. 1976. A Synthetic Benchmark. *Comput. J.* 19, 1 (Jan. 1976), 43–49. doi:10.1093/comjnl/19.1.43
- [6] Jane W. S. Liu. 2000. *Real-Time Systems*. Prentice Hall, Upper Saddle River, NJ.
- [7] L. Sha, R. Rajkumar, and J.P. Lehoczky. 1990. Priority Inheritance Protocols: An Approach to Real-Time Synchronization. *IEEE Trans. Comput.* 39, 9 (Sept. 1990), 1175–1185. doi:10.1109/12.57058